

Chapter 2

Introduction to C

Programming

An overview of C Programming Language

Chapter Targets

- Write simple C program
- Use simple input and output statement
- Use the fundamental data types
- Learn computer memory concepts
- Use arithmetic operators
- Learn the precedence of arithmetic operators
- Write simple decision-making statements
- Begin focus on secure C programming practices

A Simple C Program: Printing a Line of Text

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

```
Welcome to C!
```

Comments

- Begin with //
- Be ignored by the C compiler and do not cause any machine-language object code to be generated.
 - Do not cause the computer to perform any action when the program is run.
- Document programs and improve program readability.
 - Help other people read and understand your program.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

```
Welcome to C!
```

Comments

- You can also use `/*...*/` **multi-line comments** in which everything from `/*` on the first line to `*/` at the end of the line is a comment.
- We prefer `//` comments because they're shorter and they eliminate the common programming errors that occur with `/*...*/` comments, especially when the closing `*/` is omitted.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

```
Welcome to C!
```

#include Preprocessor Directive

- A directive to the C preprocessor
- Lines beginning with # are processed by the preprocessor before compilation.
- Tells the preprocessor to include the contents of the **standard input/output header** (<stdio.h>) in the program.
- Header contains information used by the compiler when compiling calls to standard input/output library functions such as **printf**.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

Blank Lines and White Space

- Use blank lines, space characters and tab characters to make programs easier to read.
 - Known as **white space**
- Normally ignored by the compiler

Welcome to C!

A Simple C Program: Printing a Line of Text (cont'd)

1. `// A first program in C.`
2. `#include <stdio.h>`
3. `// function main begins program execution`
4. `int main( void )`
5. `{`
6. `printf( "Welcome to C!\n" );`
7. `} // end function main`

Welcome to C!

The main Function

- Part of every C program
 - C programs contain one or more functions, one of which must be main.
- The parentheses after main indicate that main is a program building block called a **function**.
- Every program in C begins executing at the function main.
- The keyword `int` to the left of main indicates that main “returns” an integer (whole number) value.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.  
2. #include <stdio.h>  
3. // function main begins program  
   execution  
4. int main( void )  
5. {  
6.     printf( "Welcome to C!\n" );  
7. } // end function main
```

The main Function

- Can receive information when they're called upon to execute.
- The **void** in parentheses here means that main does not receive any information.

Welcome to C!



Good Programming Practice 2.1

Every function should be preceded by a comment describing the function's purpose.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.  
2. #include <stdio.h>  
3. // function main begins program  
   execution  
4. int main( void )  
5. {  
6.     printf( "Welcome to C!\n" );  
7. } // end function main
```

The main Function

- A left brace begins the body of every function, and a corresponding right brace ends the function.
- This pair of braces and the portion of the program between the braces is called a **block**.

Welcome to C!



Good Programming Practice 2.2

Add a comment to the line containing the right brace, }, that closes every function, including main.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {           Arguments   Statements Terminator
6.     printf( "Welcome to C!\n" );
7. } // end function main   Statements
```

Welcome to C!

An Output Statement

- Instructs the computer to print on the screen the string of characters marked by the quotation marks.
 - A string is known as a character string, a message or a literal.
 - “f” as **formatted**
- The characters normally print exactly as they appear between the double quotes in the printf statement.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

Escape Sequences

- The characters `\n` were not printed on the screen.
- The backslash (`\`) is called an **escape character**.
- When encountering a backslash in a string, the compiler looks ahead at the next character and combines it with the backslash to form an escape sequence.

Welcome to C!

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

Escape Sequences

- `\n`: Newline
- `\t`: Horizontal tab
- `\a`: Alert
- `\\`: Backslash
- `\"`: Double quote

Welcome to C!

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // A first program in C.
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome to C!\n" );
7. } // end function main
```

Welcome to C!

The Linker and Executables

- Standard library functions are not part of C programming language.
- Compiler provides a space in the object program for a “call” to the library function.
- Linker locates the library functions and inserts the proper calls.
 - Misspelling causes the linker spots an error.
- The linked program is called an **executable**.



Common Programming Error 2.1

Mistyping the name of the output function printf as print in a program.



Good Programming Practice 2.3

Indent the entire body of each function one level of indentation (we recommend three spaces) within the braces that define the body of the function. This indentation emphasizes the functional structure of programs and helps make them easier to read.

1. `// A first program in C.`
2. `#include <stdio.h>`
3. `// function main begins program execution`
4. `int main( void )`
5. `{`
6. `printf( "Welcome to C!\n" );`
7. `} // end function main`

Indentation

- Improve readability
 - Emphasize the structure of program
- Set convention for indent size
 - Three spaces (recommend)
 - A tab

Welcome to C!



Good Programming Practice 2.4

Set a convention for the indent size you prefer and then uniformly apply that convention. The tab key may be used to create indents, but tab stops can vary. Professional style guides often recommend using spaces rather than tabs.

A Simple C Program: Printing a Line of Text (cont'd)

```
1. // Print with two printf
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome " );
7.     printf( "to C!\n" );
8. } // end function main
```

Using Multiple `printf`s

- Each `printf` resumes printing where the previous `printf` stopped printing.

Welcome to C!

A Simple C Program: Printing a Line of Text.

```
1. // Print multiple lines with a printf
2. #include <stdio.h>
3. // function main begins program
   execution
4. int main( void )
5. {
6.     printf( "Welcome\nto\nC!\n" );
7. } // end function main
```

Using `printf` to print multiple lines

- Each time the `\n` (newline) escape sequence is encountered, output continues at the beginning of the next line.

```
Welcome
To
C!
```

Another Simple C Program: Adding Two Integers

```
1 // Addition program
2 #include <stdio.h>
3
4 // function main begins program execution
5 int main( void )
6 {
7     int integer1; // first number to be entered by user
8     int integer2; // second number to be entered by user
9
10    printf( "Enter first integer\n" ); // prompt
11    scanf( "%d", &integer1 ); // read an integer
12
13    printf( "Enter second integer\n" ); // prompt
14    scanf( "%d", &integer2 ); // read an integer
15
16    int sum; // variable in which sum will be stored
17    sum = integer1 + integer2; // assign total to sum
18
19    printf( "Sum is %d\n", sum ); // print sum
20 }
```

```
Enter first integer
45
Enter second integer
72
Sum is 117
```

Another Simple C Program: Adding Two Integers

- Variable and variable definitions

- Variable definitions

```
int integer1; // first number to be entered by user
int integer2; // second number to be entered by user
int sum; // variable in which sum will be stored
```

- The names `integer1`, `integer2` and `sum` are the **names of variables**—locations in memory where values can be stored for use by a program.
 - These definitions specify that the variables `integer1`, `integer2` and `sum` are **of type `int`**, which means that they'll **hold integer values**, i.e., whole numbers such as 7, -11, 0, 31914 and the like.

Another Simple C Program: Adding Two Integers (cont'd)

- All variables must be defined with a name and a data type before they can be used in a program.
- The preceding definitions could have been combined into a single definition statement as follows:

```
int integer1, integer2, sum;
```

- ◆ Would made it difficult to describe the variables with corresponding comments

Another Simple C Program: Adding Two Integers (cont'd)

- Identifiers and Case Sensitivity

- Any variable name in C is a valid identifier.
- An identifier is **a series of characters** consisting of letters, digits and underscores (`_`) that **does not begin with a digit**.
- C is **case sensitive**—uppercase and lowercase letters are different in C, so `a1` and `A1` are different identifiers.

Another Simple C Program: Adding Two Integers (cont'd)



Common Programming Error 2.2

Using a capital letter where a lowercase letter should be used (for example, typing `Main` instead of `main`).



Error-Prevention Tip 2.1

Avoid starting identifiers with the underscore character (`_`) to prevent conflicts with compiler-generated identifiers and standard library identifiers.



Good Programming Practice 2.5

Choosing meaningful variable names helps make a program self-documenting—that is, fewer comments are needed.



Good Programming Practice 2.6

The first letter of an identifier used as a simple variable name should be a lowercase letter. Later in the text we'll assign special significance to identifiers that begin with a capital letter and to identifiers that use all capital letters.



Good Programming Practice 2.7

Multiple-word variable names can help make a program more readable. Separate the words with underscores as in `total_commissions`, or, if you run the words together, begin each word after the first with a capital letter as in `totalCommissions`. The latter style—often called camel casing because the pattern of uppercase and lowercase letters resembles the silhouette of a camel—is preferred.

Another Simple C Program: Adding Two Integers (cont'd)

- Prompting Messages

- `printf( "Enter first integer\n" );` // prompt
- displays the literal “Enter first integer” and positions the cursor to the beginning of the next line.
- This message is called a prompt because it tells the user to take a specific action.

Another Simple C Program: Adding Two Integers (cont'd)

- The `scanf` Function and Formatted Inputs
 - Use `scanf` to obtain a value from the user

```
scanf ( "%d", &integer1 ); // read an integer
```
 - The `scanf` function reads from the standard input, which is usually the keyboard.
 - Two arguments: `"%d"` and `&integer1`
 - `"%d"`
 - ◆ Format control string, indicates the type of input data
 - ◆ Indicates that the data should be an integer (the letter d stands for “decimal integer”).
 - ◆ The `%` is treated as a special character that begins a conversion specifier.



Good Programming Practice 2.8

Place a space after each comma (,) to make programs more readable.

Another Simple C Program: Adding Two Integers (cont'd)

■ `&integer1`

- ◆ Begins with an ampersand (&)—called the **address operator** in C—followed by the variable name.
- ◆ The `&`, when combined with the variable name, tells `scanf` the location (or address) in memory at which the variable `integer1` is stored.
- ◆ The computer then stores the value that the user enters for `integer1` at that location.
- ◆ For now, just remember to precede each variable in every call to `scanf` with an ampersand.



Common Programming Error 2.4

Forgetting to precede a variable in a `scanf` statement with an ampersand (&) when that variable should, in fact, be preceded by an ampersand results in an execution-time error. On many systems, this causes a “segmentation fault” or “access violation.” Such an error occurs when a user’s program attempts to access a part of the computer’s memory to which it does not have access privileges. The precise cause of this error will be explained in Chapter 7.

Another Simple C Program: Adding Two Integers (cont'd)

- Interactive Computing

- When the computer executes the preceding `scanf`, it **waits for the user** to enter a value for variable `integer1`.
- The user responds by **typing an integer**, then **pressing** the **Enter** key to send the number to the computer.
- The computer then assigns this number, or value, to the variable `integer1`.
- Any subsequent references to `integer1` in this program will use this same value.
- Functions `printf` and `scanf` facilitate interaction between the user and the computer.
- Because this interaction resembles a dialogue, it's often called **interactive computing**.

Another Simple C Program: Adding Two Integers (cont'd)

- `printf( "Enter second integer\n" );` // prompt
 - Displays the message Enter second integer on the screen, then positions the cursor to the beginning of the next line.
 - This `printf` also **prompts** the user to take action.
- `scanf( "%d", &integer2 );` // read an integer
 - Obtains a value for variable `integer2` from the user.

Another Simple C Program: Adding Two Integers (cont'd)

- Assignment statement

- `sum = integer1 + integer2; // assign total to sum`
- Calculates the total of variables `integer1` and `integer2` and assigns the result to variable `sum` using the assignment operator `=`.
- The statement is read as, “sum gets the value of `integer1 + integer2`.” Most calculations are performed in assignments.



Good Programming Practice 2.9

Place spaces on either side of a binary operator. This makes the operator stand out and makes the program more readable.

Another Simple C Program: Adding Two Integers (cont'd)

- The `=` operator and the `+` operator are called **binary operators** because each **has two operands**.
- The `+` operator's two operands are `integer1` and `integer2`.
- The `=` operator's two operands are `sum` and the value of the expression `integer1 + integer2`.



Common Programming Error 2.3

A calculation in an assignment statement must be on the right side of the `=` operator. It's a compilation error to place a calculation on the left side of an assignment operator.

Another Simple C Program: Adding Two Integers (cont'd)

- Printing with a **Format Control String**

```
printf( "Sum is %d\n", sum ); // print sum
```

- Print the literal Sum is followed by the numerical value of variable `sum` on the screen.
- Two arguments: "Sum is %d\n" and `sum`.
 - ◆ "Sum is %d\n"
 - Format control string
 - Contains some literal characters to be displayed, and it contains the conversion specifier `%d` indicating that an integer will be printed.
 - ◆ `sum`
 - Specifies the value to be printed.
 - The conversion specifier for an integer is the same in both `printf` and `scanf`.

Another Simple C Program: Adding Two Integers.

- Calculations in `printf` Statements

- We could combine the previous two statements into the statement

```
printf( "Sum is %d\n", integer1 + integer2 );
```

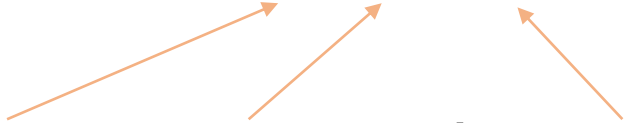


Common Programming Error 2.5

Preceding a variable included in a `printf` statement with an ampersand when, in fact, that variable should not be preceded by an ampersand.

Memory Concepts

`int a = 5;`



- Every variable has a **type**, a **name** and a **value**.
- Variable names correspond to locations in the computer's memory.

- When the statement

```
scanf( "%d", &integer1 ); // read an integer
```

is executed, the value entered by the user is placed into a memory location to which the name `integer1` has been assigned.

`integer1`

45

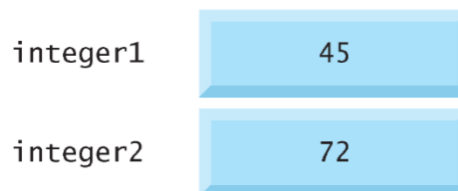


Memory Concepts (cont'd)

- Whenever a value is placed in a memory location, the value replaces the previous value in that location; thus, this process is said to be **destructive**.
- When the statement

```
scanf( "%d", &integer2 ); // read an integer
```

executes, the value is placed into location `integer2`.
- These locations are not necessarily adjacent in memory.



Memory Concepts (cont'd)

- When the statement

```
sum = integer1 + integer2; // assign total to sum
```

executes, it replaces whatever value was stored in sum.

- After sum is calculated, memory appears as

integer1	45
integer2	72
sum	117

Memory Concepts.

- The values of `integer1` and `integer2` appear exactly as they did before they were used in the calculation.
- They were used, but not destroyed, as the computer performed the calculation.
- Thus, when a value is **read** from a memory location, the process is said to be **nondestructive**.

Arithmetic in C

- Most C programs perform calculations using the C arithmetic operators.
- The arithmetic operators are all binary operators.
- For example, the expression $3 + 7$ contains the binary operator $+$ and the operands 3 and 7.

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	bm	<code>b * m</code>
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Remainder	%	$r \bmod s$	<code>r % s</code>

Arithmetic in C (cont'd)

- Integer division (/)
 - Integer division yields an integer result
 - For example, the expression $7 / 4$ evaluates to 1 and the expression $17 / 5$ evaluates to 3



Common Programming Error 2.6

An attempt to divide by zero is normally undefined on computer systems and generally results in a fatal error that causes the program to terminate immediately without having successfully performed its job. Nonfatal errors allow programs to run to completion, often producing incorrect results.

Arithmetic in C (cont'd)

- Remainder operator (%)
 - Yields the remainder after integer division
 - Can be used only with integer operands
 - The expression $x \% y$ yields the remainder after x is divided by y
 - Thus, $7 \% 4$ yields 3 and $17 \% 5$ yields 2

Arithmetic in C (cont'd)

- Arithmetic expressions in straight-line form
 - Arithmetic expressions in C must be **written in straight-line form** to facilitate entering programs into the computer.
 - Thus, expressions such as “a divided by b” must be written as **a / b** so that all operators and operands appear in a straight line.
 - The **algebraic notation $\frac{a}{b}$** is **generally not acceptable to compilers**, although some special-purpose software packages do support more natural notation for complex mathematical expressions.

Arithmetic in C (cont'd)

- Rules of operator precedence
 - C applies the operators in arithmetic expressions in a precise sequence determined by the following rules of operator precedence, which are generally the same as those in algebra:
 - Operators in expressions contained **within pairs of parentheses are evaluated first**. Parentheses are said to be at the “highest level of precedence.” In cases of nested, or embedded, parentheses, such as $((a + b) + c)$ the operators **in the innermost pair of parentheses are applied first**.

Arithmetic in C (cont'd)

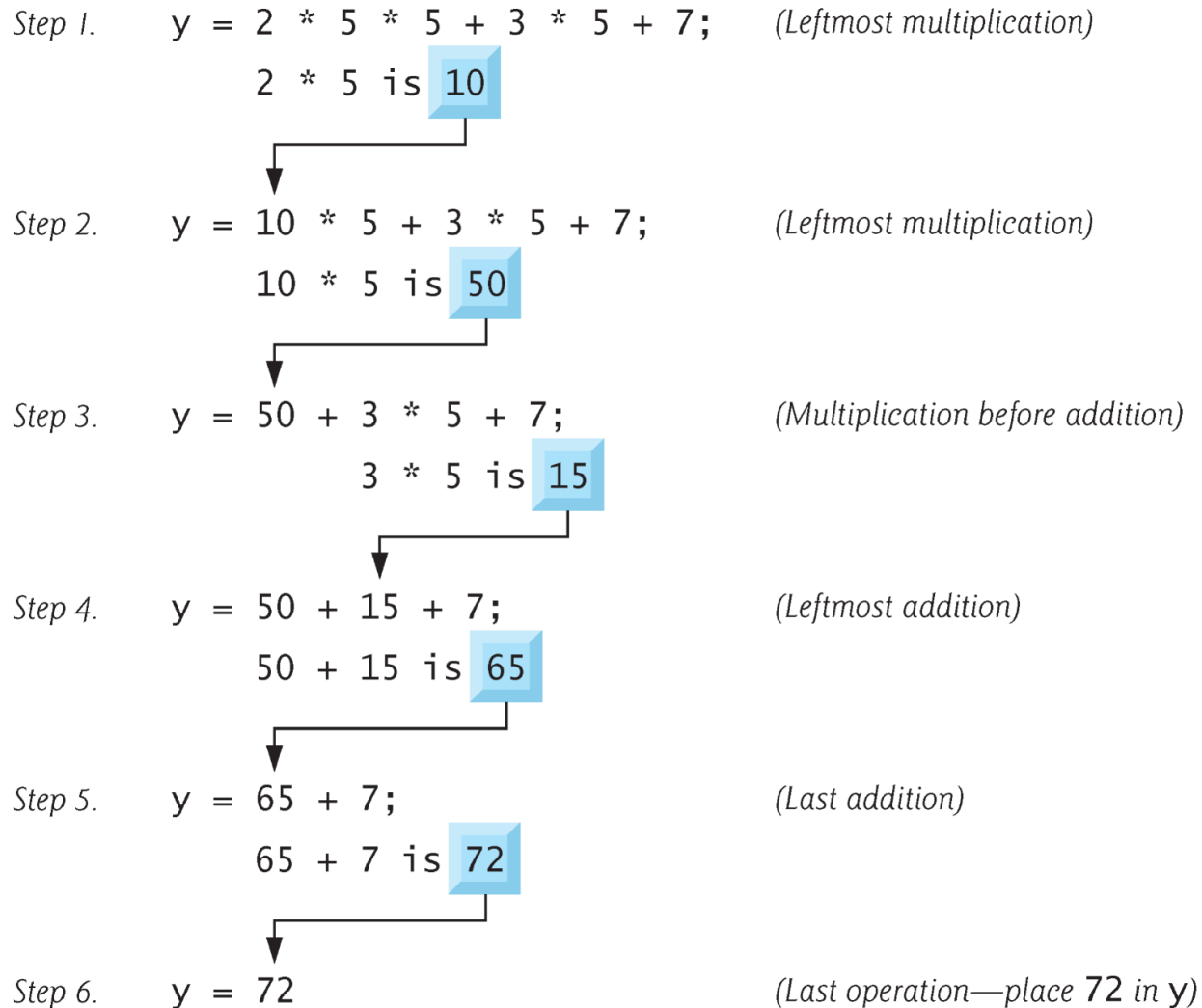
- **Multiplication, division and remainder** operations are applied next. If an expression contains several multiplication, division and remainder operations, evaluation proceeds **from left to right**. Multiplication, division and remainder are said to be on the same level of precedence.
- **Addition and subtraction** operations are evaluated next. If an expression contains several addition and subtraction operations, evaluation proceeds from **left to right**. Addition and subtraction also have the same level of precedence, which is lower than the precedence of the multiplication, division and remainder operations.
- The **assignment operator (=)** is **evaluated last**.

Arithmetic in C (cont'd)

- The rules of **operator precedence** specify the **order C uses to evaluate expressions**. When we say evaluation proceeds from left to right, we're referring to the associativity of the operators.

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the <i>innermost</i> pair is evaluated first. If there are several pairs of parentheses "on the same level" (i.e., not nested), they're evaluated left to right.
* / %	Multiplication Division Remainder	Evaluated second. If there are several, they're evaluated left to right.
+ -	Addition Subtraction	Evaluated third. If there are several, they're evaluated left to right.
=	Assignment	Evaluated last.

Arithmetic in C (cont'd)



Arithmetic in C.

- As in algebra, it's acceptable to place unnecessary parentheses in an expression to make the expression clearer.
- These are called redundant parentheses.

Decision Making: Equality and Relational Operators

- Executable C statements either perform actions or make decisions.
- We might make a decision in a program, for example, to determine whether a person's grade on an exam is greater than or equal to 60 and whether the program should print the message "Congratulations! You passed."
- This section introduces a simple version of C's if statement that allows a program to make a decision based on the truth or falsity of a statement of fact called a condition.

Decision Making: Equality and Relational Operators (cont'd)

- If the condition is met (**true**) the statement in the **body** of the if statement is **executed**.
- If the condition is *not* met (**false**) the **body** statement is **not executed**.
- Whether the body statement is executed or not, **after** the **if** statement **completes**, execution **proceeds** with the next **statement after** the **if** statement.
- **Conditions** in **if** statements are formed by using the **equality** operators and **relational operators**.

Decision Making: Equality and Relational Operators (cont'd)

Algebraic equality or relational operator	C equality or relational operator	Example of C condition	Meaning of C condition
<i>Relational operators</i>			
>	>	<code>x > y</code>	x is greater than y
<	<	<code>x < y</code>	x is less than y
≥	>=	<code>x >= y</code>	x is greater than or equal to y
≤	<=	<code>x <= y</code>	x is less than or equal to y
<i>Equality operators</i>			
=	==	<code>x == y</code>	x is equal to y
≠	!=	<code>x != y</code>	x is not equal to y



Common Programming Error 2.7

A syntax error occurs if the two symbols in any of the operators ==, !=, >= and <= are separated by spaces.

Decision Making: Equality and Relational Operators (cont'd)

```
1 // Using if statements, relational
2 // operators, and equality operators
3 #include <stdio.h>
4
5 // function main begins program execution
6 int main( void )
7 {
8     printf( "Enter two integers, and I will tell you\n" );
9     printf( "the relationships they satisfy: " );
10
11     int num1; // first number to be read from user
12     int num2; // second number to be read from user
13
14     scanf( "%d %d", &num1, &num2 ); // read two integers
15
16     if ( num1 == num2 ) {
17         printf( "%d is equal to %d\n", num1, num2 );
18     } // end if
19
20     if ( num1 != num2 ) {
21         printf( "%d is not equal to %d\n", num1, num2 );
22     } // end if
23
24     if ( num1 < num2 ) {
25         printf( "%d is less than %d\n", num1, num2 );
26     } // end if
27
28     if ( num1 > num2 ) {
29         printf( "%d is greater than %d\n", num1, num2 );
30     } // end if
31
32     if ( num1 <= num2 ) {
33         printf( "%d is less than or equal to %d\n", num1, num2 );
34     } // end if
35
36     if ( num1 >= num2 ) {
37         printf( "%d is greater than or equal to %d\n", num1, num2 );
38     } // end if
39 }
```

Enter two integers, and I will tell you
the relationships they satisfy: 3 7
3 is not equal to 7
3 is less than 7
3 is less than or equal to 7

Enter two integers, and I will tell you
the relationships they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12

Enter two integers, and I will tell you
the relationships they satisfy: 7 7
7 is equal to 7
7 is less than or equal to 7
7 is greater than or equal to 7

Decision Making: Equality and Relational Operators (cont'd)



Common Programming Error 2.8

Confusing the equality operator `==` with the assignment operator. To avoid this confusion, the equality operator should be read “double equals” and the assignment operator should be read “gets” or “is assigned the value of.” As you’ll see, confusing these operators may not cause an easy-to-recognize compilation error, but may cause extremely subtle logic errors.



Common Programming Error 2.9

Placing commas (when none are needed) between conversion specifiers in the format control string of a `scanf` statement.



Good Programming Practice 2.10

Although it’s allowed, there should be no more than one statement per line in a program.

Decision Making: Equality and Relational Operators (cont'd)

- Comparing Numbers

- Compare the values of variables num1 and num2 to test for equality.

```
if ( num1 == num2 ) {  
    printf( "%d is equal to %d\n", num1, num2 );  
}
```

- A left brace, {, begins the body of each `if` statement
- A corresponding right brace, }, ends the body
- Any number of statements can be placed in the body
- Indenting the body of each `if` statement and placing blank lines above and below each `if` statement enhances program readability.



Common Programming Error 2.10

Placing a semicolon immediately to the right of the right parenthesis after the condition in an `if` statement.

Decision Making: Equality and Relational Operators (cont'd)

- Precedence and associativity of the operators discussed so far

Operators	Associativity
()	left to right
* / %	left to right
+ -	left to right
< <= > >=	left to right
== !=	left to right
=	right to left

Decision Making: Equality and Relational Operators (cont'd)

- Some words we've used in the C programs in this chapter—in particular `int` and `if`—are **keywords** or **reserved words** of the language.
 - Words with special meaning to the C compiler
 - Careful not to use these as identifiers such as variable names

Keywords				
<code>auto</code>	<code>do</code>	<code>goto</code>	<code>signed</code>	<code>unsigned</code>
<code>break</code>	<code>double</code>	<code>if</code>	<code>sizeof</code>	<code>void</code>
<code>case</code>	<code>else</code>	<code>int</code>	<code>static</code>	<code>volatile</code>
<code>char</code>	<code>enum</code>	<code>long</code>	<code>struct</code>	<code>while</code>
<code>const</code>	<code>extern</code>	<code>register</code>	<code>switch</code>	
<code>continue</code>	<code>float</code>	<code>return</code>	<code>typedef</code>	
<code>default</code>	<code>for</code>	<code>short</code>	<code>union</code>	

Decision Making: Equality and Relational Operators.



Good Programming Practice 2.11

A lengthy statement may be spread over several lines. If a statement must be split across lines, choose breaking points that make sense (such as after a comma in a comma-separated list). If a statement is split across two or more lines, indent all subsequent lines. It's not correct to split identifiers.



Good Programming Practice 2.12

Refer to the operator precedence chart when writing expressions containing many operators. Confirm that the operators in the expression are applied in the proper order. If you're uncertain about the order of evaluation in a complex expression, use parentheses to group expressions or break the statement into several simpler statements. Be sure to observe that some of C's operators such as the assignment operator (=) associate from right to left rather than from left to right.

Exercises

- Write a complete C program to display the following content:
`Hello Midterm Exam!`
- How to write single-line and multi-line comments in your C code?
- What is the starting point to run in every C program?

Exercises

- Give the meaning of void in the parentheses of function main.
- What is the functionality of escape sequence?
- How does C compiler handle white space?
How can we use such property to improve our program?

Exercises

- Give the definition of a legal identifier.
- Give a complete C program to add three integers input by a user, and output the result to screen.
- What are the two tasks of C preprocessor?

Exercises

- What is the meaning of ampersand (&) in `scanf`?
- What is the meaning of percent sign (%) in `printf` and `scanf`?
- What are the three properties of each variable?

Exercises

- What is defined by the rules of operator precedence? What is defined by the operator associativity?